

Clase 292 — Ataques adversariales a modelos

Parte: **15 — Seguridad de IA y machine learning** · Fuente: *Goodfellow, Shlens & Szegedy, "Explaining and Harnessing Adversarial Examples" (ICLR 2015)* 🕒 Duración estimada: **120 min** · Nivel: **Intermedio**

🎯 Objetivo

Comprender por qué existen los ejemplos adversariales y aprender a generarlos, evaluarlos y defenderlos con herramientas reales. El alumno pasará de la teoría del gradiente (FGSM, PGD) a un ataque práctico sobre un clasificador propio usando Adversarial Robustness Toolbox (ART), y medirá la caída de precisión bajo ataque.

⚠️ **Ética:** los ataques de esta clase se realizan únicamente sobre modelos y datos propios en un laboratorio aislado. Atacar modelos de terceros sin autorización explícita puede ser ilegal.

📊 Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. **Explicar** por qué la linealidad local de las redes profundas produce ejemplos adversariales.
2. **Implementar** ataques FGSM y PGD contra un modelo propio con ART.
3. **Diferenciar** ataques dirigidos/no dirigidos y de caja blanca/negra.
4. **Medir** la robustez con precisión adversarial y una curva de perturbación (ϵ).
5. **Aplicar** entrenamiento adversarial como defensa y evaluar su coste.

📖 Temas

#	Tema	Por qué importa
1	Qué es un ejemplo adversarial	Perturbación mínima que cambia la predicción
2	FGSM: ataque de un paso	Base intuitiva basada en el signo del gradiente
3	PGD: ataque iterativo	Considerado el ataque de referencia (fuerte)
4	Normas de perturbación (L_∞ , L_2 , L_0)	Definen qué significa "imperceptible"
5	Ataques dirigidos vs. no dirigidos	Forzar una clase concreta o solo un error
6	Transferibilidad y caja negra	Un adversarial de un modelo suele engañar a otro
7	Ataques físicos (patches, pegatinas)	La amenaza no vive solo en píxeles digitales
8	Defensas: entrenamiento adversarial	Mejora robustez a costa de precisión limpia

Definiciones y características

- **Ejemplo adversarial:** entrada $x' = x + \delta$ con $\|\delta\| \leq \epsilon$ que causa una predicción errónea. *Característica:* δ suele ser imperceptible para un humano.
- **FGSM (Fast Gradient Sign Method):** $x' = x + \epsilon \cdot \text{sign}(\nabla_x J(\theta, x, y))$. *Característica:* un solo paso, rápido pero subóptimo.
- **PGD (Projected Gradient Descent):** FGSM iterado con proyección a la bola de radio ϵ . *Característica:* ataque fuerte, referencia para evaluar defensas.
- **Perturbación L_∞ :** limita el cambio máximo por píxel. *Característica:* la más usada en visión; ϵ típico 8/255.
- **Ataque dirigido:** fuerza una clase objetivo específica; el **no dirigido** solo busca cualquier error.
- **Transferibilidad:** un adversarial generado con un modelo sustituto engaña a otro modelo distinto; habilita ataques de caja negra.
- **Entrenamiento adversarial:** entrenar con ejemplos adversariales generados sobre la marcha (Madry et al.); mejora robustez pero reduce precisión limpia y encarece el entrenamiento.

Herramientas y preparación

```
pip install adversarial-robustness-toolbox torch torchvision numpy matplotlib
```

- **Adversarial Robustness Toolbox (ART)** — ataques y defensas listas para PyTorch/TensorFlow.
- Un modelo propio ya entrenado (p. ej. CNN pequeña sobre MNIST/CIFAR-10). Guarda sus pesos.
- GPU opcional; MNIST corre bien en CPU.

Laboratorio guiado

Todo sobre un **modelo y dataset propios**.

1. **Entrena o carga un clasificador.** Una CNN sobre MNIST alcanza >98% en test limpio. Guarda `model.pt`.

2. **Envuelve el modelo con ART.**

```
python from art.estimators.classification import PyTorchClassifier clf =  
PyTorchClassifier(model=model, loss=loss_fn, input_shape=(1,28,28), nb_classes=10,  
clip_values=(0,1))
```

3. **Mide la precisión limpia** sobre el test set y anótala como línea base.

4. **Genera FGSM.**

```
python from art.attacks.evasion import FastGradientMethod atk =  
FastGradientMethod(estimator=clf, eps=0.2) x_adv = atk.generate(x=x_test)
```

Evalúa la precisión sobre `x_adv`: debería desplomarse.

5. **Genera PGD** (`ProjectedGradientDescent`, `eps=0.2`, `max_iter=40`) y compara: PGD suele ser más efectivo que FGSM con el mismo ϵ .

6. **Barre ϵ .** Repite para $\epsilon \in \{0, 0.05, 0.1, 0.15, 0.2, 0.3\}$ y dibuja la curva precisión vs. ϵ . Observa el compromiso perceptibilidad/eficacia.
7. **Ataque dirigido.** Fuerza que un "3" se clasifique como "8" (`targeted=True`) y verifica visualmente que la imagen sigue pareciendo un 3.
8. **Defiende con entrenamiento adversarial.**

```
python from art.defences.trainer import AdversarialTrainerMadryPGD trainer =  
AdversarialTrainerMadryPGD(clf, eps=0.2) trainer.fit(x_train, y_train)
```

Reevalúa: la precisión adversarial sube, la limpia baja un poco. Documenta el trade-off.

Ejercicios

1. Explica por qué `sign()` del gradiente maximiza la pérdida bajo una restricción L_∞ .
2. Compara la precisión adversarial de FGSM vs. PGD para el mismo ϵ y explica la diferencia.
3. Genera adversariales con un modelo A y evalúalos en un modelo B distinto; mide la transferibilidad.
4. Implementa un ataque L2 (`CarliniL2Method`) y compara la magnitud de perturbación con L_∞ .
5. Muestra 5 pares (original, adversarial) y comenta si un humano notaría la diferencia.
6. Estima cuánto más tarda el entrenamiento adversarial frente al normal y discute si compensa.

Reto verificable

Entrega un **informe de robustez** de tu clasificador con: precisión limpia base, curva precisión vs. ϵ para PGD, un ejemplo dirigido exitoso, y la comparación antes/después de entrenamiento adversarial.

Criterio de aceptación: con entrenamiento adversarial, la precisión bajo PGD ($\epsilon=0.2$) mejora al menos 30 puntos porcentuales respecto al modelo sin defender, y el informe cuantifica explícitamente la caída de precisión limpia asociada. Todo el código es reproducible con `python attack.py`.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
El ataque no reduce la precisión	Falta <code>clip_values</code> o el modelo está en modo eval sin gradientes. Verifica el wrapper de ART.
Perturbaciones visibles/feas	ϵ demasiado alto. Baja ϵ y usa PGD iterativo en vez de FGSM.
"Defensa perfecta" con gradient masking	La defensa ofusca gradientes, no aporta robustez real. Evalúa con ataques de caja negra/transferencia.
Precisión limpia se hunde tras defender	ϵ de entrenamiento demasiado alto. Ajusta y busca el punto de equilibrio.
Adversariales no transfieren	Modelos muy distintos o regularización fuerte. Usa un sustituto más parecido.

? Preguntas frecuentes

? **¿Por qué existen los ejemplos adversariales?** Las redes profundas son localmente casi lineales en dimensiones muy altas; pequeñas perturbaciones alineadas con el gradiente se suman y empujan la salida sobre la frontera de decisión (Goodfellow et al.).

? **¿Sirve la detección de adversariales en lugar de la robustez?** Ayuda, pero muchos detectores se rompen ante ataques adaptativos que también engañan al detector. La robustez certificada o el entrenamiento adversarial son más sólidos.

? **¿Esto solo afecta a imágenes?** No. Hay adversariales en audio, texto (sustituciones de palabras), malware (bytes que evaden clasificadores) y datos tabulares. El principio del gradiente es el mismo.

? **¿Qué es un ataque físico?** Perturbaciones que sobreviven a la captura por cámara: pegatinas en señales de tráfico, patches impresos o monturas de gafas que engañan al reconocimiento facial.

🔗 Referencias

- Goodfellow, Shlens & Szegedy, "Explaining and Harnessing Adversarial Examples", ICLR 2015 — <https://arxiv.org/abs/1412.6572>
- Madry et al., "Towards Deep Learning Models Resistant to Adversarial Attacks", ICLR 2018 — <https://arxiv.org/abs/1706.06083>
- Carlini & Wagner, "Towards Evaluating the Robustness of Neural Networks", 2017 — <https://arxiv.org/abs/1608.04644>
- Adversarial Robustness Toolbox — <https://github.com/Trusted-AI/adversarial-robustness-toolbox>
- MITRE ATLAS, técnica "Evade ML Model" — <https://atlas.mitre.org/>

➡ Siguiente clase

Clase 293 - Envenenamiento de datos y modelos